

University of Information Technology & Sciences



Search Engine Project Report

Course Name: Data Mining and Warehouse Lab

Course Code: CSE 426

Submitted By:

Name: Md. Mahmudul Hasan

ID: 0432220005101006

Batch: 52

Section: 8A

Semester: Spring 2026

Submitted To:

Name: Mrinmoy Biswas Akash

Lecturer

Department of Computer Science
and Engineering

UITs

Submission Date: 3rd June, 2026

1. Problem Definition & Dataset

The objective of this assignment is to design and implement a specialized search engine from scratch. For this implementation, an Academic & Research Article Search Engine domain was selected.

The dataset is structured as a collection of document profiles mapped within a Python dictionary. Each document entry consists of a unique tracking numerical token (Doc ID), a structural headline (Title), and the primary text data block (Text) containing the scientific abstract content.

Explain the Challenges of Building the Search Engine

Building a specialized search engine under strict architectural constraints presents specific engineering challenges:

- **No Pre-built Libraries:** The core requirement mandates building the inverted index entirely from scratch without relying on existing enterprise index engines like Elasticsearch or Whoosh.
- **Algorithmic String Alignment:** Custom text processing pipelines must carefully filter, stem, and clean string inputs so that variations of words resolve to the same index entry without causing key errors.
- **Boolean Logic Resolution:** Correctly routing multi-word user queries through manual intersections (AND) and unions (OR) requires efficient dictionary manipulation to avoid performance bottlenecks as the dataset grows.

2. Preprocessing & Inverted Index Design

Explain Preprocessing Steps and Inverted Index Structure

The text pipeline processes raw document text through four systematic stages using allowed NLTK tools:

1. **Case Normalization:** Converts all text characters to lowercase to prevent casing mismatches.
2. **Regex Cleansing:** Strips away punctuation and non-alphanumeric symbols to isolate pure alphanumeric terms.
3. **Tokenization:** Splits the structured text string into individual word tokens based on whitespace boundaries.
4. **Stop-words Filtering & Stemming:** Filters out common English functional words (e.g., "is", "the", "and") using NLTK's stop-words corpus, then applies the algorithm to

reduce terms to their base root forms (e.g., "transforming" and "transform" resolve to the same root).

The custom Inverted Index utilizes a nested hash-map architecture:

$$\text{Index Structure: } \{term : \{doc_{id}: term_{frequency}\}\}$$

This directly maps each unique processed token to the list of document IDs where it appears, along with its specific in-document frequency.

Justify Your Design Choices and Techniques

- **Nested Dictionary Selection:** Utilizing native Python dictionaries provides an average time complexity for term lookups, ensuring highly efficient posting retrieval without third-party frameworks.
- **Linguistic Stemming:** Implementing stemming reduces index dimensionality by collapsing grammatical variants into a single term index, significantly increasing retrieval recall.
- **Frequency Tracking:** Storing raw term frequency directly inside the posting list enables immediate retrieval of local metrics required for calculating mathematical relevance scores during query phases.

3. Query Processing & Ranking

Describe How Queries are Handled and Ranked (TF-IDF)

When a multi-word query string is entered, it undergoes the exact same preprocessing pipeline (lowercase, regex clean, tokenization, stop-words removal, and stemming) to generate query tokens.

The query engine evaluates the query tokens based on the chosen mode:

- **Boolean OR:** Collects the matching document keys for all terms and performs a mathematical union operation.
- **Boolean AND:** Collects the matching document keys for all terms and performs a mathematical intersection operation to isolate documents where all terms coexist.

For all matched documents, a relevance score is computed using the Vector Space Model with TF-IDF weighting:

$$\text{\textit{TF}}$$

=

$$\frac{\text{\textit{Frequency of Term in Document}}}{\text{\textit{Total Word Count of Document}}}$$

$$\text{IDF} = \log\left(\frac{\text{Total Documents}}{\text{Document Frequency} + 1}\right)$$

$$\text{Relevance Score} = \sum (\text{TF}_t \times \text{IDF}_t)$$

Explain Trade-offs (Accuracy vs. Efficiency)

- **Index-Time Processing vs. Query-Time Speed:** Spending processing cycles to construct a detailed inverted index during the initial boot phase increases system setup time but ensures that query-time searches execute almost instantly.
- **Boolean Filtering vs. Full Vector Space Scoring:** To maximize efficiency, the system applies a hard Boolean filter (AND/OR) first to minimize the candidate document set before running the heavier mathematical TF-IDF scoring loops, optimizing response times.

4. System Analysis & Discussion

Explain Key Design Decisions and Challenges

- **Document Length Normalization:** A major design decision was dividing the raw term frequency by the total document length. Without this step, longer text entries would naturally score higher simply because they contain more words, creating a systematic bias.
- **Memory Constraints:** Because native Python data structures reside in-memory, handling exceptionally large datasets could cause memory exhaustion. For small to medium specialized databases, however, this approach maximizes processing speed.

Provide Reasoning for Your Approach and Possible Improvements

The implemented design provides an transparent framework that completely bypasses rigid third-party black-box tools. The architecture handles complex search queries cleanly while keeping resource footprints minimal.

Possible Future Improvements:

1. **BM25 Scoring Integration:** Replacing basic TF-IDF with the Okapi BM25 algorithm to implement advanced term-frequency saturation and better length normalization.
2. **Phrase and Proximity Searching:** Modifying the postings list from tracking basic frequencies to storing explicit word index positions to support exact phrase matching (e.g., searches enclosed in quotation marks).

5. Results & Conclusion

Show Sample Outputs and Summarize Findings

The custom search engine was validated using multi-word inputs across both retrieval routing modes:

--- Search Results for (OR Mode): 'machine learning healthcare' ---

Doc ID	Title	Snippet	Score
2	New Innovation in Healthcare Tech	Healthcare systems are adopting machine learning models to...	0.2054
1	Breakthroughs in Artificial Intelligence	Artificial Intelligence and machine learning are transforming...	0.1861
4	Machine Learning Models for Climate	Researchers use machine learning and artificial intelligence...	0.1733

=====

--- Search Results for (AND Mode): 'artificial intelligence climate' ---

Doc ID	Title	Snippet	Score
4	Machine Learning Models for Climate	Researchers use machine learning and artificial intelligence...	0.2432

Summary of Findings

The system successfully met all core constraints outlined in the project parameters. The OR Mode broader strategy correctly surfaced all three relevant documents containing any target vocabulary words, prioritizing the most relevant healthcare document at the top. Meanwhile, the AND Mode strict filtering accurately isolated Document 4 as the only entity containing all three target search terms simultaneously. The results confirm that building an isolated, custom-engineered indexing system using Python dictionaries is highly effective for delivering accurate, prioritized information retrieval.